

Running the GAME68 compiler on 6809/6800 and FLEX: 6802 boards

 flexonsbd.blogspot.com/2019/05/6802game68.html

Wednesday, May 8, 2019

Running the GAME68 compiler on a 6802 board.

The GAME68 compiler worked on my custom-made 6802 board.

In my previous blog post, I reported that I was able to run the GAME80 interpreter on an 8080SBC. In connection with that, I received a message from skyriver that they had successfully run the GAME80 compiler on CP/M. Therefore, I was also able to run the GAME80 compiler by referring to skyriver's blog.

This GAME80 compiler, created by Satoshi Nakajima, was ported to various machines besides the TK-80BS, but unfortunately, it wasn't ported to the 68-series for a long time. Then, in the May 1981 issue of ASCII magazine, Yoshiaki Matsushima finally published the GAME68 compiler for the H68.

Since I had used GAME3, I wanted to run this compiler, but unfortunately, I couldn't get it to work on my homemade microcomputer at the time.

However, I thought it might be possible with my current homemade 6802 board, so I decided to try porting it.

Since GAME3 already runs on my current homemade 6802 board, I manually entered the code from a saved copy of ASCII magazine. Even if you don't have a copy of ASCII magazine, the creator has made it public on their blog, so there's no need to worry. (More details below)

[1] Create the necessary files

The input files are:

the compiler source (GAME3 program format \$3E00~)

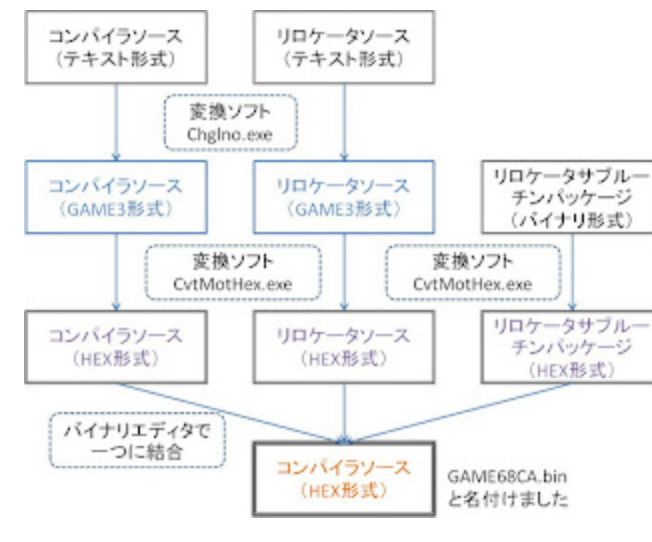
, the relocatable subroutine package (binary format \$3C80~\$3DAA)

, and the subroutine package relocater (GAME3 program format \$3C00~)

.

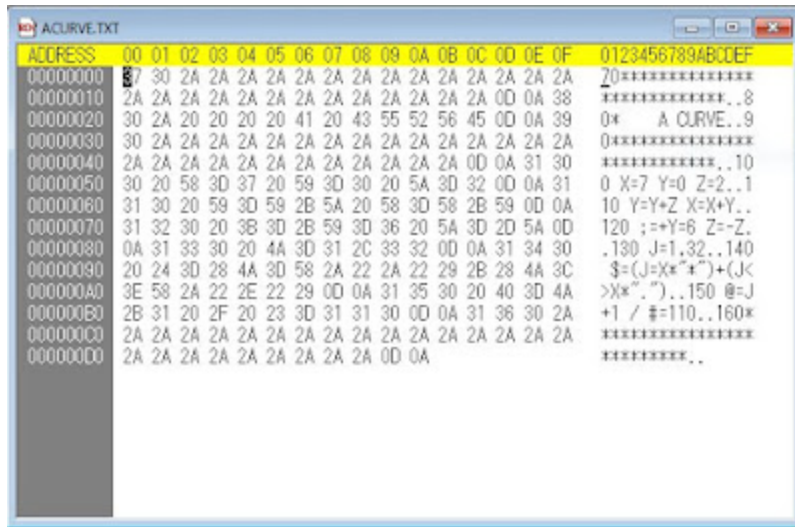
I created a single file combining these three and use it by pre-loading it along with the GAME3 interpreter. However, in memory, GAME3 programs are stored in a format where line numbers are converted to 2-byte binary, newline characters at the end of lines are converted to \$00, and \$FF is added to the end of the file. Therefore, the program listing needs to be converted beforehand.

The procedure is shown below.

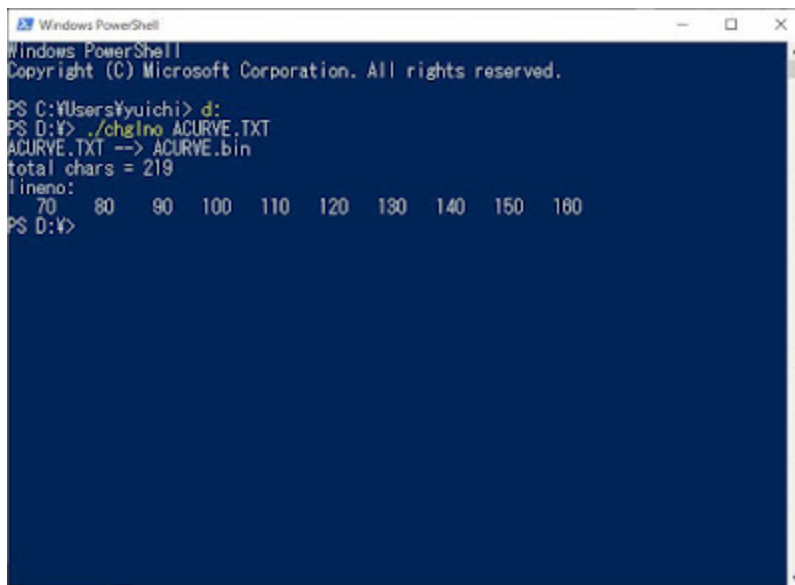


Compiler creation procedure

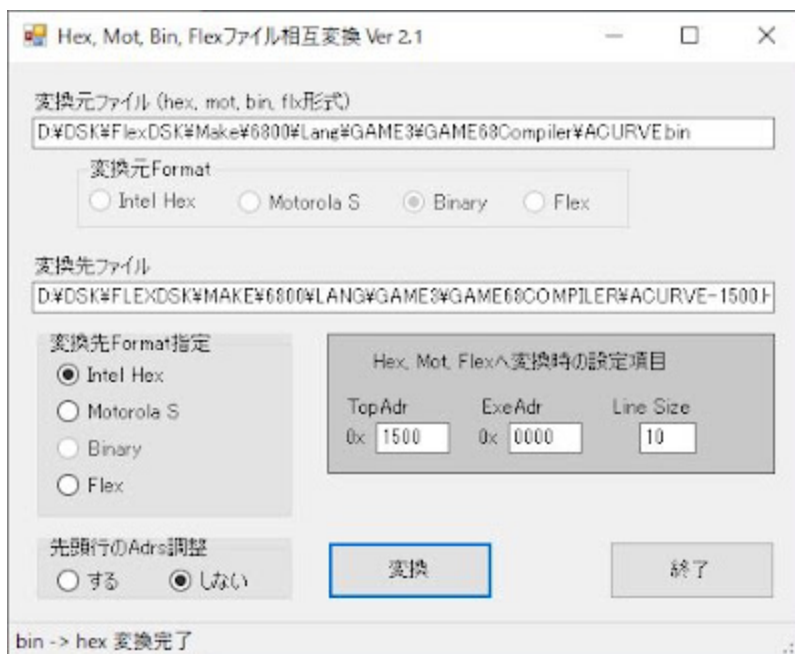
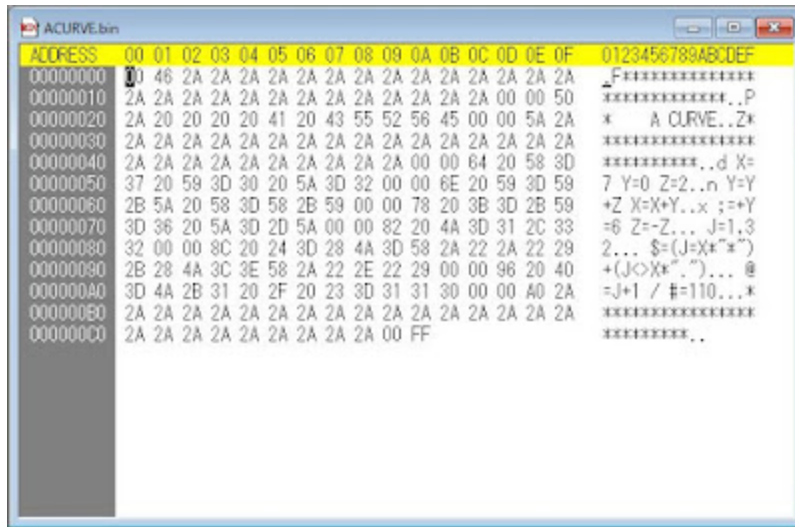
The conversion process is performed on Windows using ChgIno.exe and CvtMotHex.exe. As an example, the process of a sample program called ACURVE, provided by Mr. Onishi, the developer of the GAME3 interpreter, is shown below.



Source program before processing



The conversion is done using Chglno.exe.



[2] Compilation Procedure

- (1) Load the GAME3 interpreter and GAME68CA compiler into memory.
- (2) Start the GAME3 interpreter.
- (3) Specify the address to load the source.

Example: =\$1500

(4) Enter the program you want to compile.

You can enter it manually, but if it is processed in the same way as the compiler source,

you can load it from a file

. Example: ACURVE, the first example for GAME3

(5) Start the compiler.

Since it has been loaded initially, just specify its address \$3E00.

Specify the address as =\$3E00 and start with #=1.

(6) Proceed as described in ASCII magazine.

Enter \$1500 for GAME:,

\$2100 for SUB:,

\$222B for STAT:,

\$1900 for WORK:,

and \$2080 for STAC:

to start compilation.

(7) Move the subroutine from \$2100.

Specify the address as =\$3C00 and start with #=1.

[3] Execute the compiled machine code

The subroutine package starts at \$2100, and the execution start address is \$222B, so you can run it in GAME's direct mode with >=\$222B.

It's a bit messy, but a copy of the screen text is shown below.

```
+++0LOAD  GAME3.CMD          FLEXに保存してあるファイルをメモリに
+++0LOAD  GAME68CA.BIN       読み込んで
+++0LOAD  ACURVE.BIN
+++

M6800  MONITOR Ver6.1        いったんモニタに戻って

>G 103                       GAME3を起動 (ソースが既にメモリ中にあるのでホットスタート)

*READY
```

:= \$1500 コンパイルするプログラムのアドレスに

*READY 変更すると

:0 確かにソースが読み込まれている

```
70*****
80*      A CURVE
90*****
100 X=7 Y=0 Z=2
110 Y=Y+Z X=X+Y
120 ;+=Y=6 Z=-Z
130 J=1,32
140 $=(J=X*"")+(J<>X*"." )
150 @=J+1 / #=110
160*****
```

*READY

:= \$3E00 コンパイラのアドレスに変更して

*READY

:#=1 コンパイラを起動する

GAME: \$1500 (入力)

48 BYT (10LINE)

SUB :\$2100 (入力)

END 222A

STAT: \$222B (入力)

WORK: \$1900 (入力)

STAC: \$2080 (入力) コンパイルが始まる

70	222B	1500
80	222B	151E
90	222B	152D
100	222B	154B
110	223F	155A
120	2257	1569
130	2277	1578
140	2287	1582
150	22BC	159C

[4] In conclusion

I understand that some parts may be unclear due to insufficient explanation, but don't worry. The developer of the GAME68 compiler has actually published documentation [on their blog](#). Please refer to that for more details.

The developer even used this compiler in their work, which is amazing!

With the developer's permission, I will be posting a list of compiler source code and other files that I entered from the magazine onto [OneDrive](#). I don't think there are any input errors, so please feel free to use it. I've also included the tools I used (ChgIno.exe, CvtMotHexBin.exe).